

第4章　Verilog HDL语言基础

4.1 Verilog HDL概述

- 计算机辅助硬件电路设计；
- 出现了上百种的HDL语言；

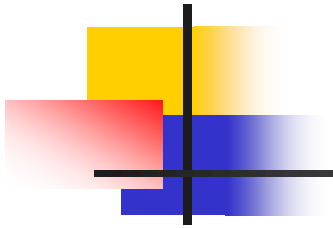
  HHDL、ISP、Verilog、BLM、AHPL、MIMOLA、SCHOLAR、TIHDL
- VHDL、Verilog HDL、System Verilog 、System C;

主要的HDL语言

- **VHDL：超高速集成电路(Very High Speed Integrated Circuit)计划中开发的硬件描述语言，1987年成为IEEE标准，后来又有1993年的IEEE版本;**
- **Verilog HDL：Gateway公司开发，后被Cadence收购，1995年成为IEEE标准。2001年，IEEE又发布了Verilog HDL的2001版本。**

Verilog HDL标准发展

- 1995年： Verilog IEEE STD 1364-1995
- 2001年： Verilog IEEE STD 1364-2001
- 2005年： Verilog IEEE STD 1364-2005
- 2005年： System Verilog IEEE 1800-2005
- 2009年： System Verilog IEEE 1800-2009
- 2012年： System Verilog IEEE 1800-2012



	VHDL	Verilog
语言风格	严谨刻板 不会出错	类似于C，书写自由，容易出错
使用部门	军方、大学	公司开发
优势	抽象建模	门级描述

VHDL和Verilog HDL

（1）两者在一定程度上可以互相转换；

（2）大多软件工具两种语言都支持；

（3）都能抽象表示电路的结构和行为，具有电路仿真和验证机制；

（4）便于文档管理、易于理解和使用。

Verilog HDL的描述层次

（1）系统级/算法级：描述系统行为和各类算法，是高层次、抽象的描述；

（2）RTL级：描述寄存器以及寄存器之间的组合逻辑关系；

（3）门级：描述逻辑门的功能和逻辑门之间的关系；

（4）开关级：描述三极管和存储节点的功能以及它们之间的连接模型。

4.2 Verilog HDL程序的基本结构

【例 5-1】 Verilog 程序的基本结构

```
module CounterHEX(CLK, RST, COUT);
  input CLK, RST;
  output [7:0] COUT;

  reg [7:0] COUTA;

  always @(posedge CLK or negedge RST)
  begin
      if (!RST) COUTA <= 0;
      else COUTA <= COUTA + 1;
  end

  assign COUT = COUTA;

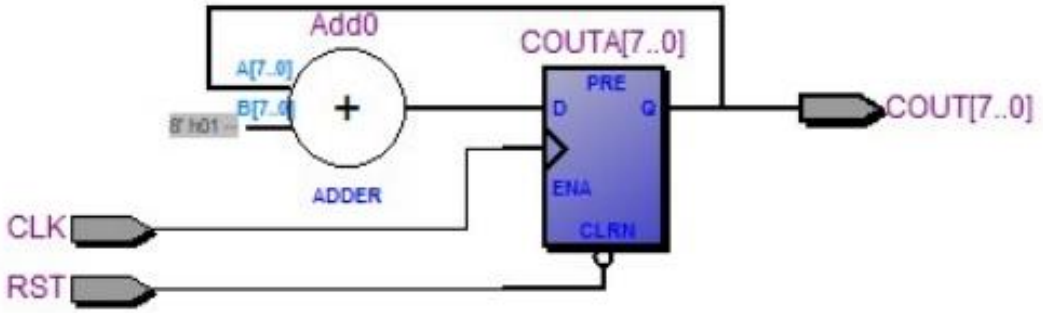
endmodule
```

端口说明

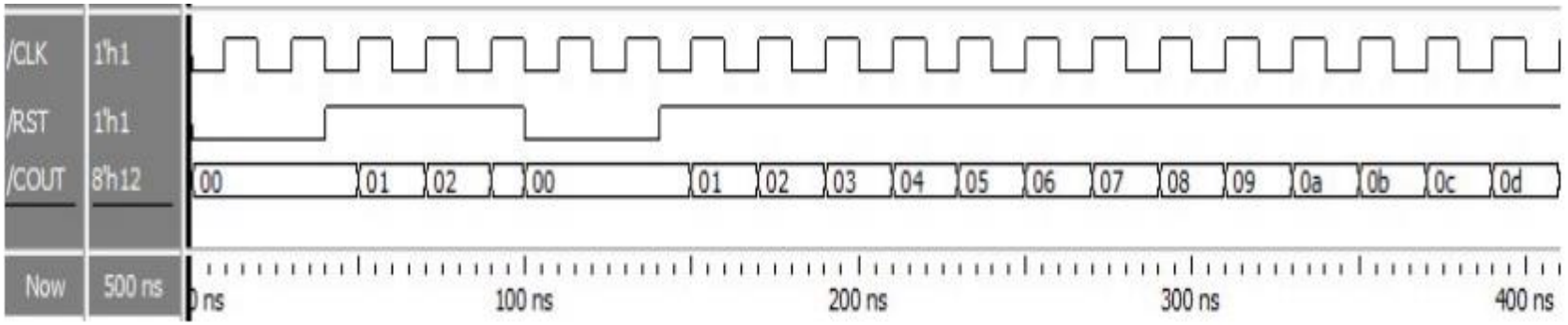
数据类型说明

电路描述语句

完整电路模块


Add0
A[7..0]
B[7..0]
8'h01
ADDER
COUTA[7..0]
PRE
D
Q
ENA
CLRN
COUT[7..0]
CLK
RST

RTL结构图


/CLK
1'h1
/RST
1'h1
/COUT
8'h12
00
01
02
00
01
02
03
04
05
06
07
08
09
0a
0b
0c
0d
Now
500 ns
0 ns
100 ns
200 ns
300 ns
400 ns

仿真图

module……endmodule结构的语法格式

module 模块名（模块端口列表）;

端口说明;

（input，output，inout）

数据类型、参数、函数等说明;

（wire，reg，parameter，function，task…）

电路描述语句;

（initial 语句，always 语句，例化语句等）

endmodule

端口说明的三种模式

- **input端口：从外部进入模块的数据通道，不能反向输出，即只能被读取，不能被赋值。语法格式为：**

input 端口名1，端口名2，……；
input [msb: lsb] 端口名1，端口名2，……；

端口说明的三种模式

- **output端口：内部数据向外输出的通道，不能反向输入，即只能被赋值，不能被读取。语法格式为：**

output 端口名1，端口名2，……；
output [msb: lsb] 端口名1，端口名2，……；

端口说明的三种模式

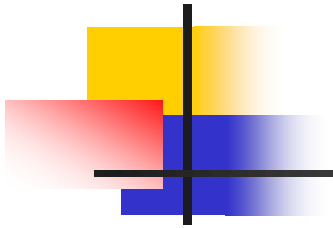
- **inout端口：数据双向传输端口，即端口可以被赋值，也可以被读取。语法格式为：**

inout 端口名1，端口名2，……；
inout [msb: lsb] 端口名1，端口名2，……；

4.2.2 Verilog HDL语言要素

1. 标识符

- 包括的符号为26个英文字母（包括大小写）、数字0~9、符号$和_（下划线）及其组合；
- 第一个符号必须是字母或下划线；
- 不能有两个连续的下划线，单一下划线前或后必须有英文字母或数字；
- 标识符中的英文字母区分大小写；
- 标识符中的字符数不能大于1024个；
- 标识符不能使用Verilog的关键字。



转义标识符

- 以反斜线“\”开头，以空白符结尾；
- 反斜线“\”之后可以包含任意可打印的字符；
- 反斜线“\”和空白符不是标识符的一部分，所以“**\OutGate**”与“**OutGate**”是一样的。

2. 空白符

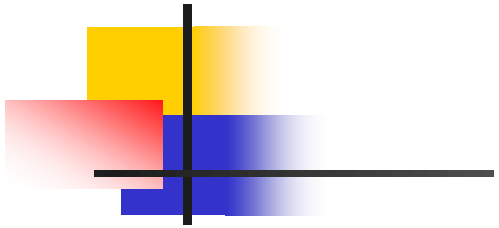
- 包括空格、制表字符和换行符；
- 仅用于分隔表示符，编译时忽略；

因此，Verilog程序可以写在一行，也可以分行书写：

```
initial begin Data = 4'b0001;  #10 Data = 4'b0010; end
```

也可以写成：

```
initial
   begin
   Data = 4'b0001;
   #10 Data = 4'b0010;
   end
```

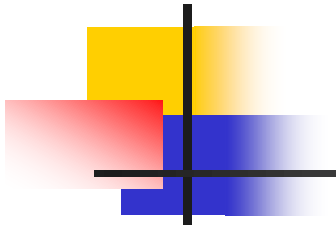


3. 关键字

- 预先定义和使用的一些有固定含义的英文单词称为关键字；
- 必须用小写字母，如**module, begin**；
- 不能当作一般标识符使用；
- 以**Verilog-2001**版关键字为主。

4. 注释

- 单行注释：采用双斜杠 / / 表示注释开始，并且注释在本行完成；
- 多行注释：跨行的注释包括在 /*…*/ 之内。



4. 数字表达方式

Verilog定义了四种基本的值：0、1、x和z，它们的含义为：

- 0：表示逻辑0、事件判别为“伪”；
- 1：表示逻辑1、事件判别为“真”；
- x/X：未知或不确定；
- z/Z：高阻态；

其他数字常量都是由以上四种值组成，分为整数、实数和字符串

（1）整数

整数的书写格式为：

- 简单的十进制格式，如16，+4，-72；
- 基数格式：

  位宽（size）′进制（base） 数值（value）

  例如：

  4′b1101

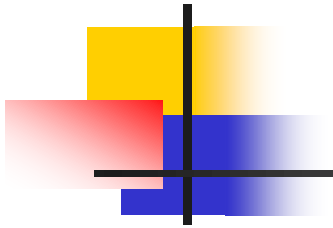
  6′o025

  8′h9A

  10′B11_0001_1011

  -6′d23

  7′Hz



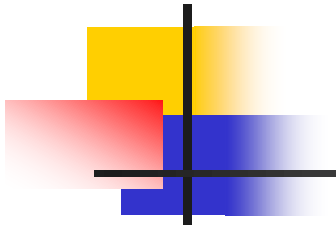
（2）实数

整数的书写格式为：

- 十进制数格式，必须带小数点，且小数点两边都有数字，如：7.0，1.54，15_000_067.584，0.1314
- 科学计数法格式，用e或E表示指数，如：

237.2e2（=23720.0）

8E-3（= 0.008）



（3）字符串

- 字符串是用双引号括起来的字符序列，如：“INTERNAL ERROR”、“ALL IS ONE”；
- 不能分成多行书写。
- 字符串主要在仿真时显示一些提示信息。
- 字符串中每个字符是用其8位ASCII码来保存，如要保存字符串“INTERNAL ERROR”，需要的位宽是14*8位。

4.3 Verilog HDL数据类型

Verilog定义了两大类数据类型：

- **线网类型： wire、tri、wor、trior、wand、triand、trireg、tri1、tri0、supply0和supply1**

- **寄存器类型：reg、integer、time、real和realtime**

4.3.1 线网类型

- **wire类型定义格式：**

  wire 变量名1，变量名2，……；

  wire [msb: lsb] 变量名1，变量名2，……；

例如：

wire DOUT，Q；

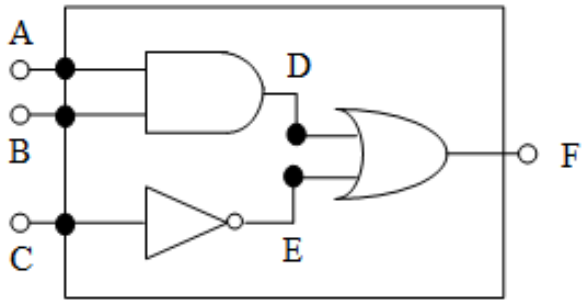
wire [3:0] Abus，AT；

wire [1:8] Databus；

线网例程

【例 5-2】 wire 类型定义

```
module myloqic(A, B, C, F);
  input A, B, C;
  output F;
  wire D, E;
  assign D = A & B;
  assign E = ~ C;
  assign F = D | E;
endmodule
```



4.3.2寄存器类型

1. reg类型定义格式：

reg 变量名1，变量名2，……；

reg [msb: lsb] 变量名1，变量名2，……；

例如：

reg AT，BT；

reg [3:0] Abus，Bbus；

多位reg类型相当于多个一位reg类型的组合，可独立使用。

Abus [3] = 1’b1；Abus [1:0] = 2’b00；

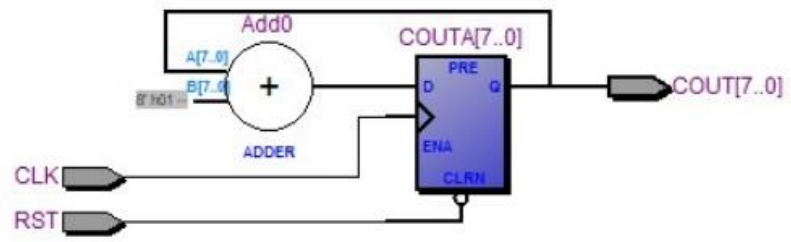
reg类型例程

【例 5-3】 输出端口重新定义为 reg 类型

```
module CounterHEX(CLK, RST, COUT);
  input CLK, RST;
  output [7:0] COUT;

  reg [7:0] COUT;   //定义 COUT 为 reg 类型

  always @(posedge CLK or negedge RST)
  begin
      if (!RST) COUT <= 0;
      else COUT <= COUT + 1;
  end
endmodule
```



2. integer类型定义格式：

integer 变量名1，变量名2，……；

integer 变量名n [msb: lsb]；

- integer没有位宽说明，默认32位；
- integer K [3:0] ；//定义了K[3]~K[0]四个整数，每个都是32位；
- integer是一个整体，不能将其中某几位单独使用；
- 给integer变量赋的值是有符号整数值；
- reg和integer类型的变量可以相互赋值，数据类型自动转换；

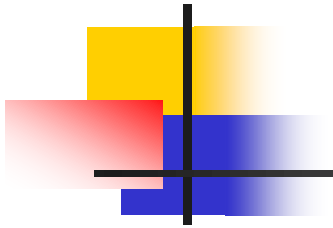
integer类型例程

【例 5-4】 integer 变量的使用

```
module MyAnd(A, B, C);
  input [7:0] A, B;
  output [7:0] C;

  reg [7:0] C;
  integer i;

  always @(A or B)
  begin
      for (i=0; i<8; i=i+1)     //i 作为循环变量
          C[i] = A[i] & B[i];
  end
endmodule
```

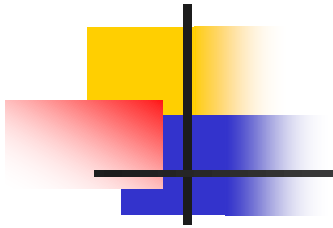


3. time类型定义格式：

time 变量名1，变量名2，……；

time 变量名n [msb: lsb]；

- time类型只能用于仿真；
- time没有位宽说明，默认64位；
- time K [3:0] ； //定义了K[3]~K[0]四个时间类型，每个都是64位；



4. real和realtime类型定义格式：

real 变量名1，变量名2，……；

realtime 变量名1，变量名2，……；

- 这两种类型只能用于仿真，不可综合；
- 是双精度浮点数，位宽为64位；

4.3.3存储器类型

存储器类型是由reg类型扩展而来，定义格式：

reg [N-1: 0] 存储器变量名 [M-1:0];

- [N-1: 0]是存储器位宽；
- [M-1: 0]是存储器容量（或深度）；
- 存储器类型定义了M个寄存器组，每个寄存器是N位。例如：

  reg [7:0] MEM [0:127];

存储器类型不能进行整体赋值，例如：

reg XROM [1: 8];

- 定义了一个容量为**8**的存储器，每个存储器单元为**1**位；
- “XROM = 8’b10010011；”是不合法的；
- 存储器类型要每个单元单独赋值，或采用文件赋值的方式，例如：

reg [7:0] mymem [0:63];

……

mymem [0] = 8’b10010011; //或者

……

$readmemb (“ram.dat”, mymem);

4.3.4参数类型

参数类型就是采用一个标识符来代表一个具体数值或字符串。定义格式：

parameter 标识符1=表达式1，标识符2=表达式2，……;

- 表达式可以是具体数值、字符串，也可以是已经定义过的标识符；
- 参数说明一般放在程序的开始处；
- 参数只能定义一次，定义之后就可以使用，例如：

parameter width = 8;

input [width-1:0] A, B;

4.4 Verilog HDL的操作符

Verilog表达式包括操作数和操作符：

操作数包括：

- 常量
- 参数
- 线网类型变量
- 寄存器类型变量
- 多位变量的位段
- 存储单元
- 函数调用

操作符包括：

- 按位逻辑操作符
- 逻辑操作符
- 缩位操作符
- 算术操作符
- 关系操作符
- 移位操作符
- 条件操作符
- 拼接操作符

4.4.1按位逻辑操作符

- 按位逻辑操作符针对操作数的每一位进行操作；
- 如果两个操作数位宽不等，按照最大位宽进行计算，位宽小的操作数左边补0；

表 5-6 按位逻辑操作符

操作符	名称	操作数	实例
~	逻辑取反	1	若 A=4'b1001, 则~A=4'b0110
&	逻辑与	2	若 A=4'b1001,B=4'b0111, 则 A&B=4'b0001
|	逻辑或	2	若 A=4'b1001,B=4'b0111, 则 A|B=4'b1111
^	逻辑异或	2	若 A=4'b1001,B=4'b0111, 则 A^B=4'b1110
~^或^~	逻辑异或非（同或）	2	若 A=4'b1001,B=4'b0111, 则 A~^B=4'b0001

按位逻辑操作结果

逻辑与 &	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

逻辑或 |	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

4.4.2逻辑操作符

- 无论操作数有多少位，逻辑操作符只在逻辑1和逻辑0上操作；
- 结果为一位逻辑1或逻辑0；
- 当操作数中有x、z时，结果都为x。

表 5-7 逻辑操作符

操作符	名称	操作数	实例
&&	逻辑与	2	若 A=4'b1001,B=4'b0000, 则 A&&B=1'b0
||	逻辑或	2	若 A=4'b1001,B=4'b0000, 则 A||B=1'b1
!	逻辑非	1	若 A=4'b1001, 则!A=1'b0

4.4.3缩位操作符

- 所有缩位操作符都是单元操作符；
- 对一个操作符从左到右进行操作，得到一位结果；

表 5-8 缩位操作符

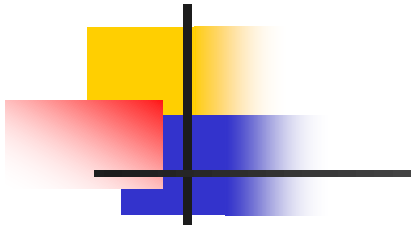
操作符	名称	操作数	实例
&	缩位与	1	若 A=4'b1001,B=4'b1111，则&A=1'b0， &B=1'b1，若某位的值为 x 或 z，则结果为 x。
~&	缩位与非	1	若 A=4'b1001,B=4'b1111，则~&A=1'b1， ~&B=1'b0
|	缩位或	1	若 A=4'b1001,B=4'b0000，则|A=1'b1， |B=1'b0，若某位的值为 x 或 z，则结果为 x。
~|	缩位或非	1	若 A=4'b1001,B=4'b0000，则~|A=1'b0， ~|B=1'b1
^	缩位异或	1	若 A=4'b1001,B=4'b0111，则^A=1'b0， ^B=1'b1，若某位的值为 x 或 z，则结果为 x。
~^或^~	缩位异或非	1	若 A=4'b1001,B=4'b0111，则~^A=1'b1， ~^B=1'b0

4.4.4算术操作符

- 加、减、乘法可以综合出电路；
- 除法、取模在除数是2的幂次方时可以综合；

表 5-9 算术操作符

操作符	名称	操作数	实例
+	加法	2	若 A=4'b1101, B=4'b0101, F 位宽为 8，则： F=A+B=8'b00010010
-	减法	2	F=A-B=8'b00001000
*	乘法	2	F=A*B=8'b01000001
/	除法	2	F=A/B=8'b00000010，取商的整数 2，小数丢弃
%	取模	2	F=A%B=8'b00000011，除法取余数 3，取第一个操作数符号



- 如果操作数的位宽不等，则表达式以最大位宽为准；
- 在赋值语句中，最终数据的位宽以目标变量的位宽为准；例如：若AT为4位，BT位5位，CT为4位，则：

  AT+BT为5位；

  CT = AT + BT；//结果为4位，高位截掉；
- 算术运算中，操作数可以是二进制数、整数及其混合类型；
- 一般而言，存储reg类型、wire类型以无符号数对待，存储integer类型以有符号数对待；

存储实例

```
reg [5:0] DATA0;
integer  DATA1;
……
DATA0 = -6d12; //赋值后，DATA0 中存储的是 110100，作为无符号数 52 看待
DATA1 = -6d12; //赋值后，DATA1 中存储的是 110100，作为有符号数-12 的补码看待
```

可以用signed明确定义reg类型、wire类型为有符号数

```
input signed [3:0] A, B; //对输入端口 A、B 设定为有符号数
reg signed data; //对 reg 变量 data 设定为有符号数
```

4.4.5关系操作符

- 分为不等操作符和相等操作符；
- 结果为“真”（逻辑1）或“假”（逻辑0）；
- 如果不等操作比较中有x或z，则结果为x；
- 位宽不等的操作数比较时，先将位宽补齐。

表 5-10 不等操作符

操作符	名称	操作数	实例
>	大于	2	15 > 60 结果为 0；4'b0110 > 4'b0101 结果为 1
<	小于	2	15 < 60 结果为 1；28 < 6'b1x0101 结果为 x；
>=	大于或等于	2	4'b110 >= 4'b0101 结果为 1
<=	小于或等于	2	4'b0111 <= 4'b0011 结果为 0

- 相等操作符“= =”比较时，按二进制位逐位比较，结果为“真”（逻辑1）或“假”（逻辑0）；

  如果操作数中包含x或z，则结果为x；

  如果位宽不一样，补齐后比较；

- 全等操作符“= = =”比较时，按二进制位逐位比较，包括x或z，如果表述完全相同，则结果为“真”；

  如果位宽不一样，直接判断为“假”；

表 5-11 相等操作符

操作符	名称	操作数	实例
==	等于	2	4'b0110==4'b110 结果为 1；2'b0x==2'b0x 结果为 x
!=	不等	2	7 != 20 结果为 1
===	全等	2	2'b0x===2'b0x 结果为 1； 4'b0110===4'b110 结果为 0；
!==	不全等	2	4'b0110!==4'b110 结果为 1

4.4.6移位操作符

- 逻辑左移和逻辑右移在空出的位置补0；
- 有符号数左移在空出的位置补0；
- 有符号数右移在空出的位置补符号位。

表 5-12 移位操作符

操作符	名称	操作数	实例
<<	逻辑左移	2	8'b01101110 << 3 结果为 8'b01110000
>>	逻辑右移	2	8'b01101110 >> 4 结果为 8'b00000110
<<<	有符号数左移	2	8'b10001010 <<< 2 结果为 8'b00101000
>>>	有符号数右移	2	8'b10001010 >>> 2 结果为 8'b11100010

4.4.7条件操作符

- 条件操作符语法格式：

  条件表达式？ *表达式1*：*表达式2*

- 条件表达式为“真”时，选择表达式1的值，否则，选择表达式2的值。例如：

  Date = (Flag > 30) ? Cur : Nex ;

4.4.8拼接操作符

- 拼接操作符用花括号{ }表示；
- 将多个操作数的二进制表达式或者其中的某些位连接成一个新的操作数。例如：

```
reg A, B;
reg [3:0] A1BUS, A2BUS;
reg [7:0] CBUS;
……
CBUS = {A1BUS, A2BUS[3], A2BUS[2], A, B};
```

- 进行连接的每个变量要有确定的位宽，拼接操作可以嵌套；

```
CBUS = {2{A1BUS}}; // 等效于{A1BUS, A1BUS}
CBUS = {4{A, B}};  // 等效于{A, B, A, B, A, B, A, B};
CBUS = {4{2'b10}}; // 等效于 10101010;
```